HACKATHON ASSIGNMENT

Build a knowledge system that knows what's true *now*

You get five downloads of a real credit union's website, taken months apart. They contradict each other. Build a system that answers members' questions with the current, correct answer and says so honestly when the website doesn't know.

WHAT YOU BUILD

A Python `solution.py` with three methods: read snapshots, answer questions, list contradictions.

WHAT YOU GET

5 website snapshots (May–Sep 2026, ~640–705 pages each), 40 practice questions, a starter solution and the scorer.

HOW YOU'RE SCORED

A larger hidden question set: accuracy 55%, conflicts found 15%, consistency 15%, speed 15%.

WHAT YOU HAND IN

A GitHub repo link. We plug your folder straight into our scorer.

1. The problem

Our assistant answers credit-union members' questions, such as "*What's the 60-month auto loan rate?*" or "*What are your lobby hours?*", using the credit union's own website. We download (crawl) that website again and again. Each download is a **snapshot**. Websites are messy:

- A rate changes, but an old page, PDF or calculator still shows the old number.
- A promotion ends, but its page is never taken down.
- One page lists the **rate**, another the **APY**, and they look like they disagree.
- A table has tiers (by balance, term or loan amount). Read it carelessly and every tier looks like a conflict.
- Old press releases and blog posts describe offers, counts and hours from years ago.

When our system combined these snapshots, it **gave members wrong or out-of-date answers**. Every question in this assignment comes from a real problem like that.

> **Your mission:** build a system that reads the snapshots in date order and answers like a careful human expert would: with the **current** value, the **right tier**, a **source**, and an honest "I don't know" when the site doesn't say.

Two real examples from the practice questions

"What's the APR for a 60-month auto loan on a 2025 model-year vehicle?"

The auto-loan rate page says `5.39%` . A vehicle-affordability calculator page that was never updated still shows `5.34%` . A good system knows which page is authoritative. Answering 5.34%, or both, scores zero.

"What was the rate on the 10-month jumbo certificate special as of May 2026?"

The May snapshots say `3.90% APY` ; by September it is 4.00%. Questions with a date ("as of…") must be answered from the website *as it stood then*, not from the latest snapshot.

2. What's in the kit

```
snapshots_zip/           the five website downloads, one zip per date (oldest first)
snapshots/README.md      what each download is and what changed between them
eval/public_eval.jsonl   40 practice questions with expected answers
starter/solution.py      a working (but deliberately naive) solution to copy from
harness/                 the scorer we will use (the same one you run)
```

You don't need to unzip anything. The first time you run the scorer, it unpacks the zips into `snapshots/` : one folder per download, each with a `manifest.json` (url, title, date) and the pages as `pages/*.md` . If you want to browse first, unzip them into `snapshots/` yourself; the scorer leaves existing folders alone.

The five snapshots

Snapshot	Date	What happened around this download
`01_2026-05-12_baseline`	12 May 2026	First full download (~640 pages). The starting point.
`02_2026-05-19_recrawl`	19 May 2026	Almost every page re-downloaded and changed. This pull caused the most new contradictions.
`03_2026-06-09_recrawl`	9 Jun 2026	Another near-full refresh. Rates moved again.
`04_2026-08-27_rebaseline`	27 Aug 2026	First download after a 10-week gap. About 50 new pages appeared.
`05_2026-09-28_latest`	28 Sep 2026	The newest download. "Current" means this one.

3. What you build

A folder containing `solution.py` with a `build()` function. It returns an object with exactly **three methods**:

Method	What it does
`ingest(snapshot)`	We call it once per snapshot, oldest first. Learn from it.
`answer(question, as_of=None)`	Answer a member's question. Without `as_of`, answer from the newest snapshot. With a date, answer as the website stood on that day.
`conflicts()`	List every place the website contradicts itself, which value you believe is right, and why.

The exact shapes live in `harness/contract.py`, and `starter/solution.py` already follows them. **Don't rename anything:** we plug your folder straight into our scorer.

```
# What your answer() returns
Answer(
    text="The 60-month rate for 2024–27 model years is 5.39% APR.",
    sources=[Source(url="https://www.dupaco.com/personal/loans/auto-loans/")],
    confident=True,          # False = "I don't know / please contact us"
    conflict_note=None,      # optional: why you picked one value over another
)

# What conflicts() returns: one entry per contradiction you found
Conflict(
    topic="60-month auto loan APR",
    values=["5.34%", "5.39%"],
    sources=[...],
    resolution="5.39%",
    reason="rate page is authoritative; calculator page is stale",
)
```

Use any approach inside: keyword search, embeddings, a knowledge graph, an LLM, plain rules, or a mix. Put extra packages in `requirements.txt`, and say in a short `NOTES.md` how your system works and which API keys (if any) it needs.

4. Check your work

From the top folder of the kit:

```
python harness/run_eval.py --submission starter --eval eval/public_eval.jsonl
```

The first run takes a few extra seconds while it unpacks the snapshots. Then it feeds all five downloads to the starter, asks the 40 practice questions and prints your score. To test your own solution, point `--submission` at your folder. You need Python 3.9 or newer.

> The starter scores about **0.09** on purpose. Watch your score go up as you improve, and read the questions you miss, not just the number.

5. How you're scored

We run a **larger, hidden question set** through the same scorer. It includes reworded versions of the practice questions, "as of" date questions, and questions about things that changed between snapshots.

Pillar	Weight	What we look for
Accuracy	55%	The current, correctly-tiered value is in your answer. **Repeating an old or wrong value scores zero for that question**, even if the right value is also there, so listing every number doesn't pay. Citing the right page earns a little extra. When the site doesn't say, the right answer is "I don't know".
Conflicts	15%	How many real contradictions you list in `conflicts()`, and whether you picked the right winner.
Consistency	15%	The same question asked in different words gets the same answer, and asking twice gives the same answer.
Speed	15%	Full marks when most answers take 2 seconds or less, falling to zero at 20 seconds.

Speed and consistency are multiplied by your accuracy: a fast or consistent system that's wrong doesn't earn those points. Loading the snapshots isn't timed for scoring, but must finish within 30 minutes.

6. A simple way to approach it

1. **Read the snapshots yourself first.** Pick five topics (a rate, a fee, branch hours…) and follow them across the dates. You'll quickly see the kinds of mess.
2. **Keep history, not just the latest copy.** Remember *when* you saw each fact and *where*.
3. **Decide what "current" means.** Is the newer page right? Does the official rates page beat a blog post or a calculator? What about an expired promotion or a 2022 press release? Write your rules down.
4. **Respect qualifiers.** "48-month" vs "60-month", "rate" vs "APY", "standard" vs "jumbo", personal vs business accounts. These are different facts, not conflicts.
5. **Be honest.** If the website doesn't say, don't guess. "It isn't listed; please contact us" is a correct answer.
6. **Run the practice set often**, and look at which questions you miss and why.

7. How to submit

- ☐ Push your work to a GitHub repo and give the organizers read access.
- ☐ Your `solution.py` with `build()` is in the repo (any folder except `starter/`).
- ☐ `requirements.txt` lists every package you need. We install it in a clean Python 3.11 environment.
- ☐ `NOTES.md` explains your approach in a few paragraphs: how you decide what's current, and which API keys it needs.
- ☐ `python harness/run_eval.py --submission <your folder> --eval eval/public_eval.jsonl` runs cleanly from a fresh clone.
- ☐ Share the repo URL (and the commit you want graded) by the deadline: [date and time to be announced].

> **We grade with our own copy of the scorer.** Changing `harness/` or `eval/` in your repo has no effect, and hard-coding answers to the practice questions won't help on the hidden set.

8. Rules

- Answer only from the snapshots provided. No live website calls.
- Each `answer` call must finish within 60 seconds.
- Any language model or library is allowed. If you use a paid API, say so in `NOTES.md`.
- The data is a real credit union's public website, shared for this event only. Don't redistribute it.